Overview
Welcome to Semantic

Core Concepts
Core Concepts
Modules
Choose the Right Module
Glossary

Guides
Architecture
Learning More
GraphRAG: Graph-Augmented Retrieval
Ingest Anything
LLM Integrations
Pipeline Builder
Multi-Agent Systems
Decision Intelligence
Context Graphs
Agent Memory
Provenance & Audit Trails
Semantic Extraction
Ontology Management
Reasoning & Rules
Export & Serialization
Deduplication & Entity Merging
MCP Server
SHACL Validation
Conflict Detection & Resolution
Change Management & Versioning
Policy Engine
Visualization
Distance Intelligence
Graph Analytics

Integrations
Agno Integration
Crewai Integration
LangChain Integration
Docling Integration
Snowflake Integration
Databricks Integration
Salesforce Integration

Graph Stores
Apache AGE Graph Store

Vector Stores
pgvector Store

FAQ
FAQ

Community
Community
Community Projects
Contributing
Governance
Citation
License

Guides

Ontology Management

Generate OWL ontologies from your knowledge graph, validate class hierarchies, infer properties, and export to Turtle, RDF/XML, or JSON-LD.

`OntologyGenerator` derives a formal OWL ontology directly from the entities and relationships already in your knowledge graph — no schema design upfront. Use it to produce a machine-readable contract for your graph's **classes** and properties, then export to Turtle, OWL/XML, or JSON-LD for SHACL validation, reasoning engines, and STIX/TAXII toolchains.

What is an Ontology?

An ontology is a formal specification of the concepts and relationships in a domain. It defines a shared vocabulary for your knowledge graph by declaring:

Classes — Types of entities in your domain. For example, `ThreatActor`, `Vulnerability`, or `Software` in cybersecurity. Classes describe what kinds of things exist.

Object Properties — Relationships between entities. Examples: `exploits` (Threat Actor exploits Vulnerability), `targets` (Malware targets Organization), or `uses` (Actor uses Tool).

Datatype Properties — Attributes with literal values. Examples: `name` (text), `severity_score` (decimal), `published_date` (date), or `ip_address` (string).

An ontology makes your knowledge graph machine-readable by specifying which relationships are valid, what types of values each property can hold, and how concepts relate to each other hierarchically.

Why Use Ontologies?

Consistency. Without an ontology, one team might use *"Threat_Actor"* while another uses *"ThreatActor"* for the same concept. Ontologies enforce a single naming convention.

Validation. Ontologies enable automatic checking — does this relationship make sense? Should a Vulnerability have a CVSS score as text or number?

Reasoning. Exporting to OWL/Turtle lets external reasoning engines (HermiT, Pellet, ELK) infer new facts. If Malware subclasses Software and HAMMERTOSS is Malware, an OWL reasoner concludes HAMMERTOSS is also Software. Semantica exports the ontology; reasoning itself runs in the external tool.

Knowledge Graph Quality. Structured schemas catch errors early and ensure new data integrates cleanly with existing entities.

When To Use / When Not To Use

Use ontologies for:

- Formal knowledge graphs with complex entity relationships
- Data integration across multiple teams or organizations
- Automated reasoning and rule-based systems
- Long-term knowledge bases that need consistency over time
- Integration with external tools that expect OWL/RDF schemas

Don't use ontologies for:

- Simple semantic search over text documents
- Lightweight RAG where vector similarity is sufficient
- Prototype development where the schema changes rapidly
- Single-use data analysis that doesn't need reusability
- Cases where the overhead exceeds the complexity of your domain

> Semantica's ontology module derives formal OWL ontologies directly from entities and relationships already in your knowledge graph — no schema design upfront. A 6-stage pipeline infers classes, builds hierarchies, maps OWL types, and serializes to Turtle. The pipeline runs in memory; you do not need a running triple store.

A Simple Example

Start with a familiar business domain to understand the mechanics before diving into cybersecurity:

```
from semantica.ontology import OntologyGenerator

# Build a simple organizational graph directly
data = {
    "entities": [
        {"id": "e-1", "name": "Alice",          "type": "Person"},
        {"id": "e-2", "name": "Bob",            "type": "Person"},
        {"id": "e-3", "name": "Carol",          "type": "Person"},
        {"id": "e-4", "name": "Acme Corporation", "type": "Company"},
        {"id": "e-5", "name": "San Francisco",  "type": "Location"},
    ],
    "relationships": [
        {"source_id": "e-1", "target_id": "e-4", "type": "works_for"},
        {"source_id": "e-2", "target_id": "e-4", "type": "works_for"},
        {"source_id": "e-1", "target_id": "e-3", "type": "reports_to"},
        {"source_id": "e-4", "target_id": "e-5", "type": "headquartered_in"},
    ],
}

generator = OntologyGenerator(
    base_uri="https://company.example.org/ontology/",
    min_occurrences=1,
)

ontology = generator.generate_ontology(
    data,
    name="OrganizationOntology",
    build_hierarchy=True,
)

# Inspect what was generated
print(f"Classes: {len(ontology.get('classes', []))}")
# Classes: 3

print("Object Properties:")
for prop in ontology.get("properties", []):
    if prop.get("type") == "object":
        domain = ", ".join(prop.get("domain", []))
        range_ = ", ".join(prop.get("range", []))
        print(f"  {prop['name']} ({domain}) → {range_})")
# works_for (Person → Company)
# reports_to (Person → Person)
# headquartered_in (Company → Location)

print("Datatype Properties:")
for prop in ontology.get("properties", []):
    if prop.get("type") == "data":
        print(f"  {prop['name']} ({prop.get('range')})")
# name (string)
```

The **`base_uri`** (`https://company.example.org/ontology/`) becomes the namespace prefix for all classes and properties. `Person` becomes `<https://company.example.org/ontology/Person>` in the exported RDF.

Object properties connect entities to other entities (`works_for` , `reports_to`) **Datatype properties** connect entities to literal values (`name`).

The graph that has no schema

Now with a schema understanding, populate a knowledge graph with CTI data to make the pipeline mechanics visible.

```
ntice.context import AgentContext, ContextGraph
ntice.vector_store import VectorStore

ectorStore(backend="faiss", dimension=768)
ontextGraph()
gentContext(vector_store=vs, knowledge_graph=graph, graph_expansion=True)

(

CVE-2024-3400 is a critical vulnerability in PAN-OS exploited by APT29.",
APT29 is a Russian state-sponsored threat actor targeting NATO governments.",
PAN-OS is a network operating system developed by Palo Alto Networks.",
HAMMERTOSS is a backdoor malware used by APT29 for command-and-control.",

ct_entities=True,
ct_relationships=True,


 point me here -0 nodes and several edges. But no formal schema.
 and a hypothetical "Lazarus Group" are both ThreatActors —
hing enforces that both must have an attribution_confidence property.
raph nodes: {len(graph.to_dict().get('nodes', []))}")
```

Generating the ontology

`OntologyGenerator` reads your graph dict and runs the 6-stage pipeline.

```
from semantica.ontology import OntologyGenerator

generator = OntologyGenerator(
    base_uri="https://cti.example.org/ontology/",
    min_occurrences=1,  # every entity type that appears at least once becomes
)

ontology = generator.generate_from_graph(
    graph.to_dict(),
    name="CyberThreatOntology",
    build_hierarchy=True,  # infer parent→child class relationships
)

# What the pipeline produced:
print(f"Classes   : {len(ontology.get('classes', []))}")
# Classes   : 4  + ThreatActor, Vulnerability, Software, Malware

print(f"Properties: {len(ontology.get('properties', []))}")
# Properties: 3  + exploits (object), targets (object), name (datatype)

# Inspect a class:
for cls in ontology.get("classes", []):
    print(f"  {cls['name']}  parent={cls.get('parent')}")
# ThreatActor   parent=None
# Vulnerability parent=None
# Software      parent=None
# Malware       parent=Software   ← hierarchy inferred because HAMMERTOSS was
```

The hierarchy entry for `Malware` shows the pipeline detected that malware is a sub-type of software based on co-occurrence patterns in the relationship graph. You can override these inferences manually before exporting.

Validating the schema

Run structural validation before exporting.

```
from semantica.ontology import validate_ontology

result = validate_ontology(ontology)

print(f"Valid: {result.get('valid', False)}")
# Valid: True

for w in result.get("warnings", []):
    print(f"  WARN : {w}")
# WARN : Class 'Malware' has no declared datatype properties

for e in result.get("errors", []):
    print(f"  ERROR: {e}")
# (none — the ontology is structurally sound)
```

A warning about missing datatype properties is common at this stage. It means the inference pipeline found the class in your graph but none of the nodes carried explicit attribute values. You can add properties manually using `ClassInferer` and `PropertyGenerator` before the next export cycle.

Growing the ontology as new node types emerge

Use `ClassInferer` to add new entity types incrementally without regenerating the full ontology.

```
from semantica.ontology import ClassInferer, PropertyGenerator

# Fine-grained control: infer classes from the new batch of entities
new_entities = [
    {"id": "kev-001", "name": "KEV-CVE-2024-3400", "type": "KEVEntry",
     "due_date": "2024-04-19", "ransomware_use": "Known"},
    {"id": "kev-002", "name": "KEV-CVE-2023-4966", "type": "KEVEntry",
     "due_date": "2023-11-14", "ransomware_use": "Unknown"},
]

inferrer = ClassInferer()
new_classes = inferrer.infer_classes(new_entities)
# [{"id": "KEVEntry", "name": "KEVEntry", "parent": None, ...}]

# Manually set the parent to Vulnerability before merging
for cls in new_classes:
    if cls["name"] == "KEVEntry":
        cls["parent"] = "https://cti.example.org/ontology/Vulnerability"

# Inspect the hierarchy
hierarchy = inferrer.build_class_hierarchy(new_classes)
print(hierarchy)
# {"KEVEntry": {"parent": "...Vulnerability", "children": []}}

# Merge into the existing ontology
ontology["classes"].extend(new_classes)

# Infer properties from the new nodes
prop_gen = PropertyGenerator()
# PropertyGenerator reads entity attributes and relationship patterns
# to produce datatype properties (due_date: xsd:date) and object properties
```

The pattern here is incremental: you run `infer_classes` on each new batch, review the results, adjust parent assignments where the pipeline guessed wrong, and merge. The ontology grows with the graph rather than falling behind it.

Generating an ontology from unstructured text

`LLMOntologyGenerator` extracts classes and properties from prose using an LLM — useful for bootstrapping when no structured graph exists yet.

```
from semantica.ontology import LLMOntologyGenerator

llm_gen = LLMOntologyGenerator(provider="groq", model="llama-3.1-8b-instant")

ontology_from_text = llm_gen.generate_ontology_from_text(
    """
    APT29 (also known as Cozy Bear) is a Russian state-sponsored threat actor.
    They use spear-phishing emails to deliver HAMMERTOSS malware.
    HAMMERTOSS communicates over Twitter and GitHub to evade detection.
    The group has been observed exploiting CVE-2024-3400 in PAN-OS appliances.
    """
)

# The LLM identified: ThreatActor, Malware, Vulnerability, Platform, Communicat
print(f"Classes extracted: {len(ontology_from_text.get('classes', []))}")

# Supported providers: "groq", "openai", "anthropic", "ollama"
```

On this page

What is an Ontology?
Why Use Ontologies?
When To Use / When Not To Use
A Simple Example
The graph that has no schema
Generating the ontology
Validating the schema
Growing the ontology as new node types emerge
Generating an ontology from unstructured text
Exporting for downstream systems
Common Pitfalls
Domain Examples
Related Guides

`LLMOntologyGenerator` is best for bootstrapping a new domain where no structured graph exists yet. Once you have a graph, prefer `OntologyGenerator.generate_from_graph()` — it is deterministic, reproducible, and does not consume LLM tokens on every run.

Exporting for downstream systems

Export in the format your downstream tools expect.

```python
from semantica.export import export_owl, export_rdf

# OWL/XML — for Protégé, OWL API, HermiT, Pellet
export_owl(ontology, "cyber_threat.owl", format="owl-xml")

# Turtle — compact, human-readable; preferred for SHACL toolchains
export_rdf(ontology, "cyber_threat.ttl", format="turtle")

# JSON-LD — for web APIs and linked-data applications
export_rdf(ontology, "cyber_threat.jsonld", format="jsonld")

# N-Triples — for bulk load into triple stores (GraphDB, Stardog, Oxigraph)
export_rdf(ontology, "cyber_threat.nt", format="ntriples")
```

The exported Turtle file is the input to Semantica's SHACL validation pipeline. See the **SHACL Validation** guide for how to generate constraint shapes from this ontology and run them against live graph data.

Common Pitfalls

Over-modeling. Don't create 50 classes when 10 would suffice. Start simple and add complexity only when you need formal distinctions for reasoning or validation. Having separate classes for `MaliciousEmail` and `PhishingEmail` is only useful if they have different properties or relationships.

Ontology drift. When new entity types appear in your graph, the ontology becomes stale unless you regenerate or incrementally update it. Set up monitoring to detect when new entity types appear that aren't covered by your current ontology.

Inconsistent class naming. Pick a convention (CamelCase, snake_case, or kebab-case) and stick to it. Mixing `ThreatActor`, `threat_actor`, and `threat-actor` in the same ontology creates confusion and breaks tooling that expects consistent naming patterns.

Domain Examples

Defense — CTI/Threat | Security — SOC/Incident | Life Science — Clinical/Pharma | Banking — R

A defense CTI team ingests raw OSINT reports each morning. The ontology must stay interoperable with STIX 2.1 and the NATO MISP taxonomy, so IRIs follow the DoD namespace and the ontology is exported to OWL/XML for the org's SIEM reasoning plugin.

```python
from semantica.context import AgentContext, ContextGraph
from semantica.vector_store import VectorStore
from semantica.ingest import ingest_file, ingest_web
from semantica.ontology import OntologyGenerator, validate_ontology
from semantica.export import export_owl, export_rdf

vs    = VectorStore(backend="faiss", dimension=768)
graph = ContextGraph()
ctx   = AgentContext(vector_store=vs, knowledge_graph=graph, graph_expansion=Tr

# Ingest a campaign PDF and NVD advisory page
cti_report = ingest_file("apt29_cozycar_2024.pdf", method="file")
nvd_entry  = ingest_web("https://nvd.nist.gov/vuln/detail/CVE-2024-3400", metho

ctx.store(
    [cti_report.text, nvd_entry.text],
    extract_entities=True,
    extract_relationships=True,
)

generator = OntologyGenerator(
    base_uri="https://ontology.dod.mil/cyber/",
    min_occurrences=1,
)
ontology = generator.generate_from_graph(
    graph.to_dict(),
    name="CyberThreatOntology",
    build_hierarchy=True,
)

result = validate_ontology(ontology)
print(f"Ontology valid: {result.get('valid')}")
# Ontology valid: True

# OWL/XML for the SIEM reasoning plugin; Turtle for the SHACL pipeline
export_owl(ontology, "./ontologies/cyber_threat.owl", format="owl-xml")
export_rdf(ontology, "./ontologies/cyber_threat.ttl", format="turtle")

print(f"Classes    : {len(ontology.get('classes', []))}")
print(f"Properties : {len(ontology.get('properties', []))}")
# Classes    : 7  (ThreatActor, Vulnerability, Malware, Platform, Campaign, ...
# Properties : 9  (exploits, targets, uses, name, cvss_score, ...)
```

Related Guides

- **SHACL Validation** — generate W3C SHACL constraint shapes from your ontology and validate live graph data against them
- **Reasoning & Rules** — apply forward/backward-chaining rules over your ontology to derive new facts
- **Export & Serialization** — export graphs to RDF, GraphML, CSV, and Neo4j Cypher
- **Semantic Extraction** — extract entities and relationships that feed ontology generation
- **Context Graphs** — the knowledge graph that ontology generation reads from